

Step 1 — Install Java (Required for Spark)

Spark runs on JVM.

Check Java:

```
java -version
```

If not installed:

Install **OpenJDK 11**

Windows (using winget):

```
winget install EclipseAdoptium.Temurin.11.JDK
```

Step 2 — Install Python

Check:

```
python --version
```

Python $\geq$ 3.9 recommended.

Step 3 — Install PySpark

Open terminal in VS Code.

```
pip install pyspark
```

Check installation:

python

Then run:

```
import pyspark
print(pyspark.__version__)
```

PART 1 — Spark Execution Plan

Before Spark executes any job it builds a **query execution plan**.

There are **three major plans**.

Plan	Description
Logical Plan	What the query intends to do
Optimized Logical Plan	Catalyst optimizes it
Physical Plan	Actual execution strategy

Example Query

```
df.filter("price > 100") \
  .groupBy("city") \
  .sum("price") \
  .show()
```

Spark internally generates:

```
Logical Plan
|
Optimized Logical Plan
|
Physical Plan
|
Execution
```

Spark Query Execution Plans

When Spark executes a query, it **does not immediately run the code**.

Instead it builds a **query plan** and then optimizes it before execution.

Spark internally generates **four stages of plans**:

Plan Type	Purpose
Parsed Logical Plan	Query interpreted from user code
Analyzed Logical Plan	Spark resolves columns and tables
Optimized Logical Plan	Catalyst optimizer improves the plan
Physical Plan	Actual execution strategy

You can view this using:

```
df.explain(True)
```

Example 1 — Simple Filter Query

Dataset:

orders

Example data:

order_id	city	price
1	NY	200
2	Chicago	50
3	NY	300
4	Dallas	400

PySpark Code

```
df = spark.read.csv("orders.csv", header=True, inferSchema=True)

df.filter(df.price > 100).show()
```

Now run:

```
df.filter(df.price > 100).explain(True)
```

Plan Generated by Spark

Spark prints something like this.

```
== Parsed Logical Plan ==
Filter (price > 100)
+- Relation[order_id, city, price]

== Analyzed Logical Plan ==
order_id:int, city:string, price:int
Filter (price > 100)
+- Relation[order_id, city, price]

== Optimized Logical Plan ==
Filter (price > 100)
+- Relation[order_id, city, price]

== Physical Plan ==
*(1) Filter (price > 100)
+- FileScan csv
```

Step-by-Step Explanation

1 Parsed Logical Plan

Spark converts your code into **logical operations**.

```
Filter(price > 100)
```

Spark doesn't yet know:

- column types
- schema validation

2 Analyzed Logical Plan

Spark verifies schema.

```
order_id:int  
city:string  
price:int
```

Now Spark confirms:

```
price exists
```

3 Optimized Logical Plan

Catalyst optimizer now applies optimizations.

Example optimizations:

- predicate pushdown
- column pruning

- constant folding

For this query there is **no major change**, so it remains the same.

4 Physical Plan

Spark decides **how the query will run on the cluster**.

Example:

```
FileScan csv  
Filter
```

Meaning:

Read file → apply filter → return results

Step 1 — Start Spark

```
from pyspark.sql import SparkSession  
  
spark = SparkSession.builder \  
    .appName("AdvancedSparkTraining") \  
    .master("local[*]") \  
    .getOrCreate()
```

Step 2 — Load Dataset

```
orders = spark.read.csv(  
    "data/orders.csv",  
    header=True,  
    inferSchema=True
```

)

Example Query

Find orders where price > 300

```
orders.filter("price > 300").show()
```

Viewing Execution Plan

```
orders.filter("price > 300").explain(True)
```

Output structure:

```
== Parsed Logical Plan ==  
Filter (price > 300)
```

```
== Analyzed Logical Plan ==  
price:int
```

```
== Optimized Logical Plan ==  
Filter (price > 300)
```

```
== Physical Plan ==  
*(1) Filter  
+- FileScan csv
```

Spark internally executes steps:

- 1 Read CSV file
- 2 Apply filter condition

3 Return results

LAB — Execution Plan Exploration

```
orders.select("city").explain(True)
```

```
orders.filter("price > 200").explain(True)
```

```
orders.groupBy("city").sum("price").explain(True)
```

LAB Questions

Question 1

Which section of the plan shows the actual execution strategy?

Answer

Physical Plan

This shows operations such as:

FileScan

Filter

HashAggregate

Exchange

Question 2

Which operation triggers shuffle in this query?

```
orders.groupBy("city").sum("price")
```

Answer

```
groupBy
```

Reason:

Grouping requires all rows with the same key to be placed in the same partition.

This requires **data movement between executors**, which is shuffle.

Spark Memory Architecture

Spark divides memory into **two main areas**.

```
Spark Memory
|
Execution Memory
Storage Memory
```

Execution Memory

Used for:

```
shuffle
sort
aggregation
join
```

Example operations:

```
groupBy
join
```

orderBy

Storage Memory

Used for storing cached data.

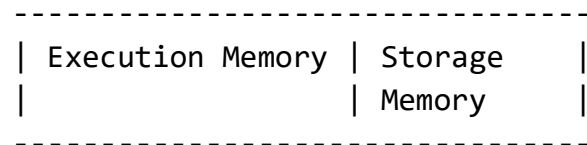
Example:

```
df.cache()
```

DataFrames stored in **Storage memory**.

Spark Memory Diagram

Executor Memory



Execution memory can **borrow from storage memory** when required.

LAB 2 — Cache Example

```
orders.cache()
```

```
orders.count()  
orders.groupBy("city").sum("price").show()
```

Question

Why caching improves performance?

Answer

Because Spark stores dataset **in memory instead of recomputing it from source**.

Shuffle Mechanics

Concept

Shuffle occurs when Spark must **redistribute data across partitions**.

Operations causing shuffle:

Operation

groupBy

join

distinct

orderBy

Example Query

```
orders.groupBy("city").sum("price").show()
```

Internal Execution Example

Assume cluster:

Executor1
Executor2
Executor3
Executor4

Initial partitions:

Executor1
NY 200
Chicago 300

Executor2
Dallas 400
NY 150

Executor3
Chicago 250
NY 350

Executor4
Dallas 500
Chicago 100

Spark must bring **same cities together**.

After shuffle:

Partition1 → NY
Partition2 → Chicago
Partition3 → Dallas

Now aggregation can occur.

LAB — Detect Shuffle

```
orders.groupBy("city").sum("price").explain()
```

Questions

Question

Which keyword indicates shuffle in execution plan?

Answer

Exchange

Example:

```
Exchange hashpartitioning(city)
```

This means Spark redistributed data across executors.

Join Execution in Spark

Load Customers

```
customers = spark.read.csv(  
    "data/customers.csv",  
    header=True,  
    inferSchema=True  
)
```

Basic Join

```
orders.join(customers, "customer_id").show()
```

Internal Execution Explanation

Assume:

orders table = 4 partitions
customers table = 2 partitions

Executors:

Executor1
Executor2
Executor3
Executor4

Initial distribution:

Executor1 → orders partition1
Executor2 → orders partition2
Executor3 → orders partition3
Executor4 → orders partition4

Customers table partitions also distributed randomly.

Spark must bring **same customer_id rows together**.

Shuffle occurs:

orders shuffled by customer_id
customers shuffled by customer_id

Now join happens.

Join Strategy Used

Spark usually chooses:

SortMergeJoin

You can confirm using:

```
orders.join(customers,"customer_id").explain(True)
```

LAB — Join Analysis

```
orders.join(customers,"customer_id").explain(True)
```

LAB Questions

Question

Which join type appears in physical plan?

Answer

Usually:

SortMergeJoin

Reason:

Both tables are treated as large tables.

Spark sorts and merges partitions.

Broadcast Join

Scenario

Tables:

orders = large table
customers = small table

Instead of shuffling both tables, Spark broadcasts small table.

Broadcast Join Execution

Driver sends small dataset to all executors.

Driver
|
Broadcast customers
|
Executors receive copy

Executors now perform join locally.

Executor1 → orders + customers
Executor2 → orders + customers
Executor3 → orders + customers
Executor4 → orders + customers

No shuffle required.

Broadcast Join Code

```
from pyspark.sql.functions import broadcast

orders.join(
    broadcast(customers),
    "customer_id"
).show()
```

LAB — Broadcast Join

```
orders.join(customers,"customer_id").explain(True)

orders.join(broadcast(customers),"customer_id").explain(True)
```

LAB Questions

Question

Which join type appears in broadcast join?

Answer

BroadcastHashJoin

Question

Why is broadcast join faster?

Answer

Because:

Small table copied to executors
Join happens locally
No network shuffle

Data Skew

Concept

Data skew happens when **some keys occur much more frequently than others.**

Example dataset:

```
customer_id
101
101
101
101
101
102
103
104
```

Partitioning by customer_id leads to imbalance:

Executor1 → 80% rows
Executor2 → 10%
Executor3 → 10%

Executor1 becomes bottleneck.

Fixing Data Skew — Salting

Add random value to distribute keys.

Example:

```
customer_id + salt
```

Salting Code

```
from pyspark.sql.functions import rand

orders.withColumn(
    "salt",
    (rand()*5).cast("int")
)
```

Adaptive Query Execution (AQE)

Why AQE Was Introduced

Before Spark 3, Spark made **all execution decisions before the job started**.

Spark would:

1. Generate execution plan
2. Decide join strategy
3. Decide number of shuffle partitions
4. Execute job

Problem:

Spark **did not know the real data size yet**.

Example scenario:

orders table = 1 TB
customers table = 5 MB

Spark might choose:

SortMergeJoin

But after filtering:

orders becomes only 100 MB

Better join strategy would be:

BroadcastHashJoin

But **Spark cannot change plan during execution (pre-Spark 3)**.

Solution: Adaptive Query Execution

Spark 3 introduced **AQE**.

AQE allows Spark to:

Change execution plan during runtime

Spark observes:

Actual data statistics
Actual partition sizes
Actual shuffle sizes

Then it **modifies execution strategy dynamically**.

AQE Architecture

Without AQE:

```
Logical Plan
  |
Physical Plan
  |
Execution
```

With AQE:

```
Logical Plan
  |
Initial Physical Plan
  |
Runtime Statistics
  |
Adaptive Optimization
  |
Modified Physical Plan
  |
Execution
```

Spark can **adjust the plan mid-execution**.

How AQE Works Internally

Spark executes first stage.

Example:

filter → shuffle → join

After shuffle completes, Spark knows:

Actual partition sizes

Actual table sizes

Then Spark **re-optimizes remaining stages**.

AQE Feature 1

Dynamic Join Selection

Spark can change join strategy **during runtime**.

Example Scenario

Tables:

orders = 1 TB

customers = 10 MB

Query:

```
orders.join(customers,"customer_id")
```

Initial Spark plan:

SortMergeJoin

But suppose filter reduces orders:

```
orders = 20 MB  
customers = 10 MB
```

Now broadcast join is possible.

AQE changes join to:

BroadcastHashJoin

Example Code

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

Run query:

```
orders.join(customers,"customer_id").explain(True)
```

You may see:

```
AdaptiveSparkPlan  
BroadcastHashJoin
```

Before AQE:

```
orders shuffle  
customers shuffle  
SortMergeJoin
```

After AQE optimization:

Broadcast customers
Join locally

Huge performance improvement.

AQE Feature 2

Shuffle Partition Coalescing

One common Spark problem:

Too many shuffle partitions

Example:

```
spark.sql.shuffle.partitions = 200
```

But dataset is small.

Example result:

200 tiny partitions

Each partition creates:

1 task

Which leads to:

task scheduling overhead

AQE Solution

AQE merges small partitions.

Example:

Before:

200 partitions

After AQE:

20 partitions

Spark automatically **coalesces partitions**.

Example Visualization

Before AQE:

|10MB|5MB|2MB|3MB|1MB| ...
200 partitions

After AQE:

|40MB|30MB|35MB|25MB|

Better task size.

AQE Feature 3

Skew Join Optimization

Data skew is very common.

Example dataset:

customer_id

1
1
1
1
1
1
1
1
2
3
4

Partitioning by customer_id creates imbalance.

Partition1 → customer_id=1 (huge)

Partition2 → customer_id=2

Partition3 → customer_id=3

Executor handling partition1 becomes **slow bottleneck**.

AQE Solution

Spark detects skewed partition.

Example:

Partition size threshold exceeded

Spark splits partition:

Before:

Partition1 → 5 GB

After:

Partition1A → 2.5 GB

Partition1B → 2.5 GB

More parallel processing.

AQE Feature 4

Dynamic Partition Pruning

Useful in **join queries with filters**.

Example:

Tables:

sales
customers

Query:

```
SELECT *  
FROM sales  
JOIN customers  
ON sales.customer_id = customers.customer_id
```

```
WHERE customers.country = 'USA'
```

Without AQE:

```
scan full sales table
```

With AQE:

Spark detects:

```
Only USA customers needed
```

Spark prunes partitions in sales table.

Less data scanned.

Enabling AQE

Spark 3 usually enables AQE by default.

But you can explicitly enable it.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

Important AQE Configurations

Enable AQE

```
spark.sql.adaptive.enabled
```

Enable skew join handling

```
spark.sql.adaptive.skewJoin.enabled
```

Enable partition coalescing

```
spark.sql.adaptive.coalescePartitions.enabled
```

Minimum partition size

```
spark.sql.adaptive.advisoryPartitionSizeInBytes
```

Lab

Dataset:

```
orders.csv  
customers.csv
```

Step 1

Enable AQE

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

Step 2

Run join query

```
orders.join(customers,"customer_id").show()
```

Step 3

Check plan

```
orders.join(customers,"customer_id").explain(True)
```

Look for:

AdaptiveSparkPlan

Lab Questions

Question 1

What does AQE allow Spark to do?

Answer:

Modify query execution plan during runtime

Question 2

Name three AQE optimizations.

Answer:

Dynamic join selection

Shuffle partition coalescing

Skew join optimization

Question 3

Why is AQE useful?

Answer:

Spark makes decisions based on actual runtime data statistics

Dataset:

```
orders = 200GB  
customers = 5MB
```

Query:

```
join orders with customers
```

Which join should Spark use?

Expected answer:

Broadcast join

AQE can **automatically detect this and switch join type.**

Summary of AQE Benefits

Benefit	Impact
Better join selection	faster joins
Reduced shuffle partitions	fewer tasks

Skew handling	balanced workload
Dynamic optimization	smarter execution

1. Serialization in Spark

Concept

Serialization is the process of **converting objects into a format that can be transferred across the network or stored on disk.**

Spark uses serialization when:

- Sending data between executors
- Writing shuffle data
- Caching objects
- Transmitting tasks from driver to executors

Why Serialization Matters

Poor serialization can cause:

large network traffic
slow job execution
high memory usage

Spark Serialization Options

Spark supports two serializers.

Serializer	Description
------------	-------------

Java Serializer	Default serializer
Kryo Serializer	Faster and more compact

Java Serializer

Default option.

Pros:

- simple
- no configuration required

Cons:

- slow
- large serialized objects

Kryo Serializer

Faster and more efficient.

Advantages:

- 10x faster
- smaller object size
- less network overhead

Enable Kryo Serializer

```
spark.conf.set(  
    "spark.serializer",  
    "org.apache.spark.serializer.KryoSerializer"
```

)

Exercise

Question

Why is Kryo preferred in large Spark jobs?

Answer

Because it produces **smaller serialized objects and faster serialization**.

2. Spill (Disk Spill)

Concept

Spill happens when **Spark runs out of memory during execution**.

Spark temporarily writes data to disk.

When Spill Happens

Spill typically occurs during:

shuffle
sort
aggregation
joins

Example query:

```
orders.groupBy("city").sum("price")
```

Large shuffle operations may exceed memory.

Spark then:

```
writes intermediate data to disk
```

Internal Flow

```
Executor memory full
```

```
|
```

```
Spark writes data to disk
```

```
|
```

```
Later reads it back
```

This slows execution.

Detecting Spill

You can observe spill in:

- Spark UI
- Stage metrics

Metrics include:

```
memory spill
```

```
disk spill
```

Exercise

Question

What is the main reason spill slows Spark jobs?

Answer

Because disk I/O is **much slower than memory operations**.

3. Spark Task Scheduling

Spark scheduler assigns tasks to executors.

Scheduling Components

Component	Role
Driver	creates tasks
Cluster Manager	allocates resources
Executors	run tasks

Scheduling Process

Driver → Task Scheduler → Executors

Task Scheduling Strategies

Spark uses:

FIFO scheduling
Fair scheduling

FIFO

First job submitted executes first.

Fair Scheduling

Resources shared across jobs.

Exercise

Question

Which scheduler allows multiple jobs to share resources?

Answer

Fair Scheduler

4. Spark Checkpointing

Checkpointing saves intermediate results to **reliable storage**.

Used in:

- long lineage pipelines
- streaming jobs

Why Checkpointing

Spark lineage can become very long.

Example pipeline:

df1 → df2 → df3 → df4 → df5

If failure occurs:

Spark must recompute entire lineage.

Checkpointing stores:

df3

Then recomputation starts from there.

Example

```
spark.sparkContext.setCheckpointDir("/tmp/checkpoints")
```

```
df.checkpoint()
```

Exercise

Question

When should checkpointing be used?

Answer

When lineage is **very long or recomputation is expensive**.

5. Spark Broadcast Variables

Broadcast variables distribute **read-only data** to executors.

Example:

small lookup tables
configuration data

Example

```
broadcast_var = spark.sparkContext.broadcast([1,2,3])
```

Executors can access broadcast data efficiently.

Why Broadcast Variables Matter

Without broadcast:

data sent repeatedly to executors

With broadcast:

data sent once and reused

6. Accumulators

Accumulators allow executors to **update shared variables**.

Used for:

counters
metrics
error tracking

Example

```
acc = spark.sparkContext.accumulator(0)
```

```
def count_rows(row):  
    global acc  
    acc += 1
```

Exercise

Question

Where are accumulator values stored?

Answer

Driver

Executors update them, but driver stores final value.

7. Spark Small Files Problem

Very common in production pipelines.

Example:

millions of small files

Problems:

slow metadata lookup
slow job startup
inefficient scanning

Solution

Use:

file compaction
coalesce
optimize jobs

Example:

```
df.coalesce(10).write.parquet("output")
```

8. Spark Executor Configuration

Spark performance heavily depends on executor configuration.

Important parameters:

Config	Purpose
spark.executor.memory	executor memory
spark.executor.cores	cores per executor
spark.driver.memory	driver memory

Example

```
spark.executor.memory = 4g  
spark.executor.cores = 2
```

9. Data Locality

Spark tries to run tasks **where data is located**.

Levels of locality:

Locality	Description
PROCESS_LOCAL	data in same executor
NODE_LOCAL	data on same node
RACK_LOCAL	same rack
ANY	anywhere in cluster

Better locality → faster execution.

10. Spark Fault Tolerance

Spark is fault tolerant through **lineage**.

Example:

```
RDD1 → RDD2 → RDD3
```

If partition lost:

Spark recomputes using lineage.

Final Exercise

Question 1

Dataset sizes:

```
orders = 500GB  
customers = 20MB
```

Which join strategy should be used?

Answer

Broadcast Join

Because small table can be broadcast.

Question 2

Query:

```
groupBy(customer_id)
```

But 90% rows contain:

```
customer_id = 101
```

What problem occurs?

Answer

Data Skew

Question 3

Which operation usually triggers shuffle?

Answer

```
groupBy  
join  
distinct  
orderBy
```

FINAL LAB — End-to-End Pipeline

Step 1 — Load datasets

```
orders  
customers  
products
```

Step 2 — Join datasets

```
df = orders.join(customers,"customer_id") \  
            .join(products,"product_id")
```

Step 3 — Revenue by city

```
df.groupBy("city").sum("price")
```

Step 4 — Top customers

```
df.groupBy("customer_id") \
  .sum("price") \
  .orderBy("sum(price)", ascending=False)
```

Step 5 — Save output

```
df.write.parquet("output/sales_summary")
```